Appendix: Artifact Description/Artifact Evaluation

Artifact Description (AD)

I. Overview of Contributions and Artifacts

A. Paper's Main Contributions

- C_1 OCEAN is a software emulation framework for CXL 3.0 that implements GFAM, MH-SLD for as many as 16 hosts, Dynamic Capacity Devices, fabric management, active latency injection, token-bucket bandwidth control, and MESI-based software cache coherence with distributed snoop filters.
- C_2 OCEAN provides a scalable multi-host fabric architecture with configurable CXL switches, topology discovery and address allocation, dynamic capacity management, and SHM, TCP, and RDMA transports.
- C_3 OCEAN exposes emulated CXL memory through the Linux CXL, NUMA, and DAX interfaces, allowing existing applications and system software to run without source changes.
- C_4 OCEAN is evaluated against real CXL Type-3 hardware using LMbench, STREAM, and OSU microbenchmarks. It is then used to study hardware cache-coherence coverage with Tigon/TPC-C and YCSB, twelve memory-placement policies with GROMACS, and calibration of the inter-host LogP timing model.

B. Computational Artifacts

- A_1 OCEAN source code, modified QEMU and workload submodules, guest image and kernel, experiment scripts, and plotting scripts. Development repository: https://github.com/cxl-emu/OCEAN. Archived artifact: **TODO: insert the Zenodo DOI and immutable version or commit hash used for evaluation**.

Artifact	Contributions	Reproducible paper elements
A_1	C_1–C_4	Functional CXL device and multi-host deployment; Figs. 3–5 (LMbench, STREAM, and OSU fidelity)

II. Artifact Identification

A. Computational Artifact A_1

Relation To Contributions

Artifact A_1 contains OCEAN's C/C++ implementation, the modified QEMU device model, host and network setup scripts, workload integrations, and analysis scripts. The core implementation supports C_1–C_3; the microbenchmark, application, and calibration workflows support C_4. The artifact also contains representative logs and plots under `OCEAN/artifact/`. The figure-oriented reproducibility workflow for Figs. 3–5 is under `artifacts/fig3`, `artifacts/fig4`, and `artifacts/fig5`.

Expected Results

After deployment, each guest should enumerate a CXL Type-3 device and expose a DAX device such as `/dev/dax0.0`. Two or more guests should attach to the same OCEAN memory server and access the shared memory pool through SHM, TCP, or RDMA. The evaluation should establish the following outcomes:

- latency grows as the LMbench working set exceeds the cache hierarchy, and OCEAN/SHM follows the large-working-set trend of real CXL hardware;
- per-host STREAM bandwidth decreases as more hosts contend for the shared pool, while OCEAN/RDMA follows real-hardware OSU Allgather latency more closely than OCEAN/TCP;
- increasing hardware-coherence coverage substantially improves TPC-C NewOrder throughput, and software-coherent protocols degrade for write-heavy YCSB mixes;
- the twelve GROMACS placement policies produce a measurable execution time spread (up to 7% in the paper), with SHM faster than TCP; and
- calibrated LogP parameters reproduce measured point-to-point latency and contention trends substantially better than the defaults.

Exact numerical agreement is not expected across different processors, networks, and CXL devices. Reviewers should evaluate the qualitative trends and the tolerances stated for each experiment below.

Expected Reproduction Time (in Minutes)

Artifact setup downloads and builds QEMU and a Linux kernel and therefore is expected to dominate preparation time. Full application experiments may take hours; a reduced smoke-test path should be supplied for functional review. Fresh measurements will be reproduced on Chameleon Cloud. The artifact will record the lease, node hardware type, image, network configuration, and observed wall time for every stage rather than estimating those values before the reproduction campaign. Regenerating and validating Figs. 3–5 from the archived CSV files takes less than one minute.

Artifact Setup (incl. Inputs)

Hardware: The baseline deployment requires an x86-64 Linux host with hardware virtualization (KVM), enough cores and memory for at least two QEMU guests, and sufficient disk space for one image per guest. Root access is required to create bridges/TAP devices, launch QEMU with KVM, and configure DAX/NUMA resources. SHM and TCP evaluation can run on one physical server. RDMA evaluation requires RDMA-capable NICs and a configured RDMA fabric. Hardware validation and LogP calibration require two hosts connected to shared CXL Type-3 memory through a CXL switch.

The reproducibility campaign for Figs. 3–5 will use reserved bare-metal nodes on Chameleon Cloud. Each run manifest

will identify the Chameleon hardware type, image, lease, node allocation, and network configuration.

The paper's reference hardware consists of two compute nodes, each with two Intel Xeon Gold 5420+ sockets (28 cores/socket) and 1 TB DRAM, connected via PCIe 5.0 to an FPGA-based CXL memory system providing a 32 GB shared pool. **TODO: state the minimum RAM, disk capacity, RDMA NIC model/link rate, and whether reviewers will receive access to the physical CXL testbed**.

Software: The paper uses QEMU 10.0 and a Linux 6.16-rc7 guest kernel. The repository build uses CMake and C++20 and depends on GCC/G++ 13, Clang/LLVM, libbpf, libcxxopts, Boost, fmt, spdlog, RDMA CM/libibverbs, libpmem, and the standard QEMU build dependencies. Python analysis requires NumPy, pandas, Matplotlib, and SciPy where used. Workload-specific requirements include OpenMPI, LMbench, STREAM, OSU Micro-Benchmarks 7.4, Tigon, YCSB, and GROMACS 2026.1.

The setup script targets Ubuntu and installs its dependencies with APT. **TODO: record the tested host distribution/version, exact kernel, Python, OpenMPI, compiler, and library versions; freeze every Git submodule revision**.

Datasets / Inputs: LMbench, STREAM, and OSU generate synthetic inputs. The latency experiment uses a 64-byte stride and array sizes from 48 MB through 2048 MB. STREAM runs Copy, Scale, Add, and Triad while varying the host count from one to ten. OSU runs MPI Allgather over powers-of-two message sizes.

Tigon uses TPC-C NewOrder and YCSB mixes on two nodes under the concurrency protocols stated in the paper. GROMACS uses the PEPSIN input and twelve placement policies. The LogP calibration benchmark operates on shared DAX memory from two MPI ranks and uses streaming groups of 512 cache lines and $N \in \{1, 2, 4, 8\}$ shared locks for contention fitting. **TODO: archive the exact TPC-C scale factor/database image, YCSB records and operation count, PEPSIN `.tpr` input, random seeds, warm-up policy, and repetitions for every experiment**.

Installation and Deployment: Clone the archived artifact recursively and verify its revision:

```
git clone --recursive <ARTIFACT-URL> OCEAN
cd OCEAN
git checkout <ARTIFACT-COMMIT>
git submodule update --init --recursive
git status --short
```

The last command should print nothing. On a dedicated Ubuntu host, install and build the dependencies and modified QEMU:

```
bash script/setup_host.sh
cmake -S . -B build \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_CXX_COMPILER=g++-13
cmake --build build -j"$(nproc)"
ctest --test-dir build --output-on-failure
```

Configure a two-guest network on one machine with `bash script/setup_network.sh 2`; use `setup_optional_cross_machine_network.sh` for multiple physical hosts. Download the archived guest kernel and disk image, make a private copy for each guest, and then start the memory server and guests as described in the artifact README.

Because setup scripts modify host networking and install system packages, they must be run only on a dedicated evaluation machine. The repository README's mutable download links must be replaced by DOI-hosted files with checksums before evaluation. **TODO: insert exact archive filenames, SHA-256 sums, and noninteractive commands for preparing and launching all guest images**.

Artifact Execution

The complete workflow is

$$\begin{aligned} T_1\ (\text{build}) &\to T_2\ (\text{network/images}) \\ &\to T_3\ (\text{server/VMs}) \\ &\to T_4\ (\text{verify DAX}) \\ &\to T_5\ (\text{experiments}) \\ &\to T_6\ (\text{plots/checks}). \end{aligned}$$

In T_1, build OCEAN, QEMU, the guest kernel/modules, and workloads. In T_2, create the bridge/TAP topology and one copy of the guest image per VM. In T_3, launch the OCEAN memory server before the guests. In T_4, verify device enumeration, DAX, NUMA topology, and guest-to-guest connectivity. In T_5, run each baseline and OCEAN configuration with identical workload parameters, including all repetitions. In T_6, parse raw logs, report the mean and dispersion, regenerate the plots, and compare the stated invariants.

All raw outputs must be written to a new result directory rather than overwriting the reference results. Record the commit hash, submodule hashes, host/guest software versions, CPU topology, and the exact command line with each run. For Figs. 3–5, the top-level driver supports independent figure selection:

```
./run-experiments.sh --figures 3,4,5
```

The default command regenerates and validates the graphs from archived CSV files. Passing `--mode collect --data results` invokes the figure-specific fresh-measurement wrapper.

Artifact Analysis (incl. Outputs)

A successful setup produces OCEAN executables under `OCEAN/build/`, launches the memory server without fatal errors, boots every guest, exposes `/dev/dax0.0` (or a documented equivalent) in each guest, and permits cross-guest communication. The current figure workflow creates raw, machine-readable results plus PDF and PNG plots for Figs. 3–5. Reference CSV files are retained under each `artifacts/figN/data/` directory, and each figure has a validation script that returns nonzero when required outputs are

absent or a stated invariant fails. The Fig. 4 reference CSV is explicitly marked as digitized from the submitted raster graph and must be replaced by the original aggregated STREAM data before the artifact is archived.

Artifact Evaluation (AE)

A. Computational Artifact A_1

Build and multi-host functional smoke test.

Artifact Setup (incl. Inputs)

Follow the installation and deployment procedure for A_1. The smoke test requires one x86-64 KVM host, two guest images, and the SHM transport; no RDMA or physical CXL hardware is required. Start the memory server with a 1 GB pool, launch two guests, and retain the server and guest console logs. **TODO: replace the descriptive launch step with exact commands matching the frozen QEMU scripts and image filenames**.

Artifact Execution

Inside each guest, run:

```
lspci | grep -i cxl
ls -l /dev/dax*
numactl --hardware
ping -c 4 <OTHER-GUEST-IP>
```

Map the same DAX-backed test region in both guests, write a deterministic cache-line pattern from guest 0, read and verify it from guest 1, then reverse writer and reader. Finally, run the repository's DAX litmus tests for message passing, store buffering, atomicity, and tearing.

```
ctest --test-dir build --output-on-failure
```

TODO: add a single script that performs the cross-guest pattern test and litmus suite and documents the number of iterations.

Artifact Analysis (incl. Outputs)

Pass criteria are: both guests enumerate the emulated Type-3 device; each has a DAX device and the expected NUMA node; all pings succeed; both directions of the shared-memory pattern test return zero mismatches; and every enabled CTest and litmus test passes. The result demonstrates the functional portions of C_1–C_3, but does not by itself establish performance fidelity.

B. Computational Artifact A_2

Microbenchmark fidelity (Figs. 3–5).

Artifact Setup (incl. Inputs)

Use the reference VM deployment. LMbench needs one guest. STREAM uses one through ten guests sharing one OCEAN pool. OSU Allgather uses two guests and is evaluated with the SHM/TCP/RDMA configurations stated in the paper; physical-CXL results may be reproduced only on the supplied CXL testbed.

Artifact Execution

For LMbench, run `lat_mem_rd` with 64-byte stride at array sizes 48, 64, 96, 128, 192, 256, 384, 512, 1024, 1536, and 2048 MB. For STREAM, run Copy, Scale, Add, and Triad at host counts 1, 2, 4, 6, 8, and 10 and report per-host MB/s. For OSU, set `CXL_DAX_PATH=/dev/dax0.0`, preload the MPI CXL shim, and run `osu_allgather` over its powers-of-two message sizes. Run the same number of warm-ups and measured repetitions for every configuration. The supplied collectors default to three LMbench warm-ups and five measured invocations for each configuration. They retain every raw stdout log and plot the arithmetic mean.

Install Matplotlib and recreate the submitted graphs from archived CSV files:

```
python3 -m pip install -r requirements.txt
./run-experiments.sh --figures 3,4,5 \
  --mode all --data reference
```

For fresh measurements, configure the benchmark-specific runner described in each figure README and execute, for example:

```
export LMBENCH_LAT_MEM_RD=/path/to/lat_mem_rd
./run-experiments.sh --figures 3 \
  --mode collect --data results
./run-experiments.sh --figures 3 \
  --mode all --data results
```

Figure 4 uses `STREAM_RUNNER`, an executable that accepts a host count and prints standard STREAM result lines. Figure 5 uses `OSU_RUNNER`, an executable that accepts `rdma`, `tcp`, or `real` and prints standard OSU result rows. These runners isolate site-specific VM and network orchestration from the common parsing, aggregation, plotting, and validation logic.

Artifact Analysis (incl. Outputs)

The evaluator should confirm three invariants: (1) LMbench latency increases as the working set leaves cache and the OCEAN/SHM large-array curve follows the real-CXL trend; (2) per-host STREAM bandwidth declines monotonically or near-monotonically as host count increases; and (3) OCEAN/RDMA is closer to the real-CXL OSU curve than OCEAN/TCP. For small OSU messages, the paper reports OCEAN/RDMA within approximately $2\times$ of hardware and TCP at approximately $3\times$ hardware. Deviations should be explained using the recorded host and network configuration.

C. Computational Artifact A_3

Hardware-coherence case study (Figs. 6–7).

Artifact Setup (incl. Inputs)

Deploy two OCEAN nodes and Tigon. Use the archived TPC-C database and YCSB inputs. Pin software versions, CPU allocation, database scale, client count, and memory capacity across configurations. Clear or restore database state between runs as required by the workload protocol.

Artifact Execution

Run TPC-C NewOrder with 2PL while sweeping hardware-cache-coherence coverage from 0% through 100%. Run YCSB while sweeping the write fraction from 0% through 100% for Tigon, DS2PL+, and Sundial+. Use identical warm-up, measurement, and repetition counts at each point; report per-node or aggregate throughput consistently. **TODO: add exact commands and state all sweep increments, database/client parameters, repetitions, and confidence interval**.

Artifact Analysis (incl. Outputs)

For Fig. 6, throughput should rise strongly with HCC coverage; the reference result rises from 12.3K txn/s at 0% to 138K txn/s at 100%, approximately $11\times$. For Fig. 7, Tigon should remain comparatively stable as writes increase, while DS2PL+ and Sundial+ should degrade sharply beyond roughly 40% writes. These trends support C_4 by isolating the cost of software cache flushes in write-heavy, cross-host transactional workloads.

D. Computational Artifact A_4

GROMACS placement-policy sweep (Fig. 8).

Artifact Setup (incl. Inputs)

Use the two-node OCEAN deployment, the archived PEPSIN input, GROMACS 2026.1, and the MPI CXL shim. Hold the number of ranks, threads, steps, CPU binding, memory capacity, and transport configuration fixed across policies.

Artifact Execution

Run both SHM and TCP for Baseline, Interleave, NUMA, frequency migration, page-table-aware placement, frequency caching, heat-aware placement, Hybrid, Locality, FIFO caching, Hugepage, Lifetime, and LoadBalance. Collect execution time after warm-up and plot the mean and dispersion for every policy/transport pair. **TODO: reconcile the paper's statement of "twelve" policies with the thirteen labels visible in Fig. 8, then provide the exact commands, repetitions, and plotting script**.

Artifact Analysis (incl. Outputs)

The reference baseline is 1.083 s. SHM should be faster than TCP for each policy, and policy choice should create a spread of as much as approximately 7% between the best and worst reference configurations. The precise ranking may depend on the host; the central result is a repeatable, nonzero policy sensitivity under otherwise identical conditions.

E. Computational Artifact A_5

LogP calibration (Fig. 9 and Table II).

Artifact Setup (incl. Inputs)

This evaluation requires two physical hosts mapping the same CXL Type-3 DAX device through a CXL switch, MPI, cache-flush instructions, and the `cxl_switch_lock_bench_mpi` calibration program. **TODO: include this program in the frozen artifact and document the CXL device, switch, firmware, DAX configuration, CPU affinity, and clock source**.

Artifact Execution

Measure sender overhead o_s with `clflushopt+sfence`, receiver overhead o_r with `clflush+mfence+load`, propagation latency L with cross-host ping-pong, and gap g with 512-line streaming flushes. Compute $L = (RTT - 2o_s - 2o_r)/2$ and $L_{hop} = L/2$. Fit contention while both hosts operate on $N \in \{1, 2, 4, 8\}$ shared CAS locks. Apply the fitted parameters to OCEAN, rerun the latency and contention tests, and regenerate Fig. 9 and Table II. **TODO: state repetitions, outlier handling, fit procedure, and exact commands**.

Artifact Analysis (incl. Outputs)

The reference fitted values are $o_s = 18$ ns, $o_r = 438$ ns, $L = 142$ ns, $g = 4.6$ ns, and 13.9 GB/s. The calibrated point-to-point latency should be within 5% of measured hardware and the contention model within 10% across the tested utilization range. The receiver overhead should dominate the latency decomposition, and calibration should visibly improve both the point-to-point and contention fits relative to the defaults.